

数学分析 II 中 Hessian 半正定二阶必要条件的 Lean 形式化

目录

1	引言	2
2	数学背景与主定理	2
3	从数学表述到 Lean 表述	5
4	核心定义与代码实现	6
4.1	局部极小点	6
4.2	Hessian 算子	6
4.3	算子的半正定性	6
4.4	Hessian 半正定性	6
5	形式化设计与证明处理	7
5.1	为何采用 Hessian operator 而非 Hessian matrix	7
5.2	为何提供两个主定理版本	7
5.3	为何独立封装一元二阶必要条件	8
5.4	证明结构的模块化分解	9
6	辅助引理与证明结构	9
6.1	直线限制函数	10
6.2	局部极小沿直线保持	10
6.3	沿直线的一阶导数与梯度	10
6.4	Hessian 二次型的表示	11
6.5	一元二阶必要条件	11

目录	2
7 主定理的形式化证明	12
7.1 显式梯度场版本	12
7.2 直接使用 <code>gradient f</code> 的版本	13
8 与传统 Hessian 矩阵的关系	13
9 结论	13
A Lean 完整代码	15
A.1 根模块 <code>Hessian.lean</code>	15
A.2 主形式化文件 <code>Hessian/Basic.lean</code>	15

摘要

二阶必要条件是多元函数极值理论的基本结论之一：若光滑函数在局部极小点处的梯度为零，则其 Hessian 矩阵必为半正定。本文在 Lean 4 证明助手中，基于 mathlib 数学库，完成了该定理的完整形式化证明。由于 mathlib 的微分学框架建立在 Fréchet 导数与内积空间之上，本文将经典教材中“二阶偏导数矩阵”的表述等价地转化为“梯度映射的 Fréchet 导数”，并以线性算子的半正定性替代矩阵半正定性。证明采用直线限制法将多元问题化为一元函数的极值问题，并通过一元二阶必要条件和中值定理完成最终推导。文章详细给出了从数学定义到 Lean 代码的术语转换表，系统讨论了形式化过程中的核心设计决策，并展示了如何将传统证明模块化地分解为可独立验证的辅助引理。本文涉及的 Lean 文件在项目所使用的 Lean 4 与 mathlib 环境下通过编译，主证明未使用 `sorry`、`admit` 或 `axiom`。

关键词：Lean 4；mathlib；Hessian 矩阵；二阶必要条件；形式化证明；数学分析 II

1 引言

在数学分析 II（多元微分学）课程中，函数极值的二阶必要条件是一个核心结论：设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 在点 x_0 处取得局部极小值，若 f 在 x_0 处二阶连续可微且梯度为零，则 Hessian 矩阵 $(\frac{\partial^2 f}{\partial x_i \partial x_j}(x_0))$ 必为半正定。该定理常用于优化理论及相关应用问题中，其标准证明通过方向导数和中值定理完成。

随着交互式定理证明器的发展，将经典数学结论形式化已成为一项有意义的实践。Lean 4 是一款基于依赖类型论的证明助手，其社区维护的数学库 mathlib 提供了分析学、代数学等领域的丰富形式化基础。然而，mathlib 中的微分学采用 Fréchet 导数的泛函分析语言：函数在某点的导数被表示为连续线性映射，而梯度则是该线性映射在内积空间中的 Riesz 表示。因此，课堂中直接使用二阶偏导数矩阵的定义无法直接移植到 mathlib 中，必须将 Hessian 重新表述为梯度函数的导数。

本文报告了利用 Lean 4 对 Hessian 半正定二阶必要条件进行形式化的全过程。我们给出了一个基于直线限制法和一元二阶必要条件的完整证明，严格遵循 mathlib 的分析 API，并提供了与传统矩阵形式的对应关系。本文给出的 Lean 代码均已在项目环境中通过编译。本文的目的在于展示如何将课堂上的数学分析证明适配至现代证明助手的规范框架之下。

2 数学背景与主定理

本节回顾经典数学分析 II 中的相关定义与定理，所有表述均采用 $\mathbb{R}^n$ 上的多元微分学语言。

设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$, $x_0 \in \mathbb{R}^n$ 。记向量空间的标准内积为 $u \cdot v = u^T v$, 范数为 $|u| = \sqrt{u \cdot u}$ 。

定义 2.1 (局部极小点) 称 x_0 是 f 的一个局部极小点, 若存在 $\varepsilon > 0$, 使得对所有满足 $|y - x_0| < \varepsilon$ 的 y , 均有 $f(x_0) \leq f(y)$ 。

定义 2.2 (梯度) 若 f 在 x_0 处可微, 则存在唯一向量 $\nabla f(x_0) \in \mathbb{R}^n$, 使得对任意方向 $v \in \mathbb{R}^n$, 方向导数

$$\lim_{t \rightarrow 0} \frac{f(x_0 + tv) - f(x_0)}{t} = \nabla f(x_0) \cdot v.$$

称 $\nabla f(x_0)$ 为 f 在 x_0 处的梯度。

定义 2.3 (Hessian 矩阵) 若梯度函数 $\nabla f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 在 x_0 处可微, 则其 Jacobi 矩阵

$$H_f(x_0) = D(\nabla f)(x_0) = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}(x_0) \right)_{n \times n}$$

称为 f 在 x_0 处的 Hessian 矩阵。

定义 2.4 (矩阵半正定) 称实对称矩阵 A 为半正定, 若对任意 $v \in \mathbb{R}^n$ 均有 $v^T A v \geq 0$ 。

定理 2.1 (Hessian 半正定的二阶必要条件) 设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 在 x_0 处取得局部极小值, 且满足

- f 在 x_0 处可微, 且 $\nabla f(x_0) = 0$;
- f 在 x_0 的某邻域内可微 (从而梯度在该邻域内存在);
- 梯度函数 $\nabla f(x)$ 在 x_0 处可微。

则 Hessian 矩阵 $H_f(x_0)$ 是半正定的, 即对一切 $v \in \mathbb{R}^n$ 有 $v^T H_f(x_0) v \geq 0$ 。

此定理的经典证明思路为: 任取方向 v , 考虑一元函数 $\varphi(t) = f(x_0 + tv)$, 由局部极小性知 $t = 0$ 是 φ 的局部极小点。利用链式法则得 $\varphi'(t) = \nabla f(x_0 + tv) \cdot v$, 故 $\varphi'(0) = 0$ 。进一步, 由 ∇f 的可微性有 $\varphi''(0) = v^T H_f(x_0) v$ 。此时将一元函数的二阶必要条件应用于 φ , 即得 $\varphi''(0) \geq 0$ 。由 v 的任意性完成证明。这一思路也是本文 Lean 形式化的数学基础。

完整数学证明 下面给出定理 2.1 的详细证明, 完全在 $\mathbb{R}^n$ 上完成。

证明 任取 $v \in \mathbb{R}^n \setminus \{0\}$ ($v = 0$ 时结论平凡成立)。构造一元辅助函数

$$\varphi(t) = f(x_0 + tv), \quad t \in \mathbb{R}.$$

第一步： φ 在 $t = 0$ 处局部极小。因 x_0 是 f 的局部极小点，存在 $\varepsilon > 0$ 使得当 $|y - x_0| < \varepsilon$ 时 $f(x_0) \leq f(y)$ 。取 $\delta = \frac{\varepsilon}{|v| + 1} > 0$ ，则当 $|t| < \delta$ 时，

$$|(x_0 + tv) - x_0| = |t||v| < \delta(|v| + 1) < \varepsilon,$$

故 $f(x_0) \leq f(x_0 + tv)$ ，即 $\varphi(0) \leq \varphi(t)$ 。因此 $t = 0$ 是 φ 的局部极小点。

第二步： φ 在 $t = 0$ 附近可导且 $\varphi'(0) = 0$ 。由条件 2， f 在 $x_0 + tv$ 处可微对充分小的 t 成立。根据方向导数的链式法则，

$$\varphi'(t) = \nabla f(x_0 + tv) \cdot v,$$

其中“ $\cdot$ ”为 $\mathbb{R}^n$ 中的标准点积。特别地，令 $t = 0$ 并利用条件 1 $\nabla f(x_0) = 0$ ，得

$$\varphi'(0) = \nabla f(x_0) \cdot v = 0.$$

第三步： φ' 在 $t = 0$ 处可导，且 $\varphi''(0) = v^\top H_f(x_0)v$ 。令向量值函数 $g(x) = \nabla f(x)$ ，则 $g: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 在 x_0 处可微，其 Jacobi 矩阵为 $H = H_f(x_0)$ 。由可微性定义，

$$g(x_0 + tv) = g(x_0) + H(tv) + o(t) = g(x_0) + tHv + o(t), \quad t \rightarrow 0.$$

其中 $o(t)$ 是 $\mathbb{R}^n$ 中满足 $\lim_{t \rightarrow 0} \frac{|o(t)|}{|t|} = 0$ 的向量。两边同时点乘 v ，得到

$$\varphi'(t) = g(x_0 + tv) \cdot v = g(x_0) \cdot v + t(Hv) \cdot v + o(t).$$

同时 $\varphi'(0) = g(x_0) \cdot v$ 。因此

$$\frac{\varphi'(t) - \varphi'(0)}{t} = (Hv) \cdot v + \frac{o(t)}{t}.$$

令 $t \rightarrow 0$ ，得

$$\varphi''(0) = (Hv) \cdot v = v^\top Hv.$$

故 φ' 在 0 处可导，二阶导数值恰为 $v^\top Hv$ 。

第四步：一元二阶必要条件推出 $v^\top Hv \geq 0$ 。 此时一元函数 φ 满足： φ 在 $t = 0$ 处局部极小； φ 在 0 的某邻域内可导，且 $\varphi'(0) = 0$ ； φ' 在 0 处可导，记 $l = \varphi''(0) = v^\top Hv$ 。下面证明 $l \geq 0$ 。

假设 $l < 0$ 。由导数定义，

$$\lim_{t \rightarrow 0} \frac{\varphi'(t)}{t} = l < 0.$$

根据极限的保号性，存在 $\delta_1 > 0$ ，使得当 $0 < t < \delta_1$ 时 $\frac{\varphi'(t)}{t} < 0$ ，从而 $\varphi'(t) < 0$ 。取 b 满足 $0 < b < \min\{\delta_1, \varepsilon\}$ ，其中 ε 为第一步中的极小邻域半径，则 φ 在 $[0, b]$ 上连续，在 $(0, b)$ 内可导。由 Lagrange 中值定理，存在 $c \in (0, b)$ 使得

$$\frac{\varphi(b) - \varphi(0)}{b - 0} = \varphi'(c).$$

因为 $0 < c < b < \delta_1$, 故 $\varphi'(c) < 0$, 于是 $\varphi(b) - \varphi(0) < 0$, 即 $\varphi(b) < \varphi(0)$ 。然而 $0 < b < \varepsilon$ 保证了 $|b| < \varepsilon$, 由局部极小性应有 $\varphi(0) \leq \varphi(b)$, 矛盾。所以 $l \geq 0$, 即 $v^\top H v \geq 0$ 。

第五步：方向任意性得到矩阵半正定。 上述推理对任意非零 $v \in \mathbb{R}^n$ 成立, 而当 $v = 0$ 时 $v^\top H v = 0 \geq 0$ 。因此

$$\forall v \in \mathbb{R}^n, \quad v^\top H_f(x_0) v \geq 0,$$

即 Hessian 矩阵 $H_f(x_0)$ 半正定。证毕。

3 从数学表述到 Lean 表述

在 Lean 的 mathlib 库中, 微分学并不直接采用 $\mathbb{R}^n$ 上的 Jacobi 矩阵, 而是以泛函分析语言统一处理赋范向量空间上的 Fréchet 导数。因此, 课堂上的许多概念在形式化时需经过一次“翻译”。本节给出关键术语的对应关系, 以帮助理解 Lean 代码。

mathlib 中使用的空间类型为一般的实内积空间 E , 满足 NormedAddCommGroup E 、InnerProductSpace $\mathbb{R} E$ 和 CompleteSpace E 。在具体应用时可视为 $\mathbb{R}^n$ 。所有映射的导数均以连续线性映射 ($E \rightarrow L[\mathbb{R}] F$) 表示。

表 1: 表 3.1 数学概念与 Lean 表达对照

数学分析概念	Lean/mathlib 中的表达
函数 $f : \mathbb{R}^n \rightarrow \mathbb{R}$	<code>f : E → ℝ</code>
点 x_0	<code>x₀ : E</code>
内积 $u \cdot v$	<code>inner ℝ u v</code>
可微性与梯度 $\nabla f(x)$	<code>HasGradientAt f (g x) x</code> 或 <code>gradient f x</code>
梯度向量场 $x \mapsto \nabla f(x)$	<code>fun x ⇒ gradient f x</code>
Fréchet 导数 (即 Jacobi 对应的线性映射)	<code>fderiv ℝ f x</code>
Hessian 矩阵 $H_f(x)$	<code>fderiv ℝ (fun y ⇒ gradient f y) x</code> , 类型 <code>E → L[ℝ] E</code>
二次型 $v^\top H v$	<code>inner ℝ (H v) v</code>
算子半正定 $\forall v, v^\top H v \geq 0$	<code>∀ v, 0 ≤ inner ℝ (H v) v</code>
局部极小点	自定义 <code>localMinimumAt f x₀</code>
梯度为零	<code>HasGradientAt f 0 x₀</code>
梯度函数可微且导数为 A	<code>HasFDerivAt (fun x ⇒ gradient f x) A x₀</code>

从表中可看出, 核心改变在于将矩阵 H 视为作用于 v 的线性算子 A , 并将 $v^\top H v$ 写成内积形式。这种表述实际上与数学分析中的思路一致: 在 $\mathbb{R}^n$ 上, 线性算子的伴随

矩阵乘以内积即得二次型。因此，形式化过程中的“泛函分析外衣”并未改变证明的本质，只是用一种更统一的语言重新描述了同样的对象。

4 核心定义与代码实现

本节给出 Lean 中与定理直接相关的几个核心定义，并解释其数学含义。

4.1 局部极小点

```
def localMinimumAt (f : E → ℝ) (x₀ : E) : Prop :=
  ∃ ε > 0, ∀ y : E, dist y x₀ < ε → f x₀ ≤ f y
```

该定义直接对应定义 2.1，使用距离函数 `dist` 描述邻域。

4.2 Hessian 算子

```
noncomputable def hessianOp (f : E → ℝ) (x : E) : E → L[ℝ] E :=
  fderiv ℝ (fun y => gradient f y) x
```

这一定义实现了“Hessian = 梯度映射的 Fréchet 导数”。这里使用 `noncomputable`，是因为 `fderiv` 和 `gradient` 在 `mathlib` 中通常作为非可计算对象定义。需要注意的是，在缺少可微性假设时，`fderiv` 与 `gradient` 仍有默认值；因此，主定理中还需要显式加入 `HasGradientAt` 与 `HasFDerivAt` 假设，以保证这些对象具有预期的数学含义。

4.3 算子的半正定性

```
def posSemidefOp (A : E → L[ℝ] E) : Prop :=
  ∀ v : E, 0 ≤ inner ℝ (A v) v
```

对于任意连续线性算子 A ，该定义断言其满足半正定条件，恰好对应 $v^T A v \geq 0$ 。

4.4 Hessian 半正定性

```
def hessianPosSemidefAt (f : E → ℝ) (x : E) : Prop :=
  posSemidefOp (hessianOp f x)
```

该定义将 Hessian 算子与半正定性相结合，直接表达“ f 在 x 处的 Hessian 半正定”这一命题。

以上四个定义构成了整个形式化的概念骨架。接下来的所有辅助引理和主定理均在此基础上展开。

5 形式化设计与证明处理

将数学分析 II 中的经典定理移植到 Lean/mathlib 中，并非简单地将纸面证明逐行翻译。本节讨论形式化过程中遇到的四个核心设计问题：Hessian 的表述形式、梯度场的假设方式、一元引理的独立封装，以及证明结构的模块化分解。

5.1 为何采用 Hessian operator 而非 Hessian matrix

课堂中 Hessian 被定义为二阶偏导数构成的矩阵 $H_f(x_0) = (\partial_{ij} f(x_0))_{n \times n}$ 。在 mathlib 中，多元函数的导数以 Fréchet 导数（即连续线性映射）统一处理，并不直接依赖坐标或偏导数索引。

若直接在 Lean 中定义矩阵版本的 Hessian，需要：

- 选定一组基并显式引入坐标函数；
- 为每个矩阵元素单独证明其等于对应的二阶偏导数；
- 处理矩阵乘法与二次型 $v^T H v$ 的计算。

然而，本定理的核心结论仅涉及二次型 $v^T H v$ 的非负性。因此，本文选择先证明 operator 版本：将 Hessian 定义为梯度映射的 Fréchet 导数，类型为 $E \rightarrow L[\mathbb{R}] E$ ，半正定性通过内积表达为 $\forall v, 0 \leq \text{inner } \mathbb{R} (A v) v$ 。这一路径具有以下优势：

- 与 mathlib API 自然对齐：fderiv 和 gradient 的现有定理可直接使用，无需额外建立矩阵层。
- 坐标无关：证明不依赖具体坐标选取；在本文语境下，可将其理解为 $\mathbb{R}^n$ 上 Hessian 矩阵对应的线性算子。
- 矩阵表示可后恢复：在需要时，通过 hessianMatrixOfOp 可在标准基下恢复传统 Hessian 矩阵（见第 8 节）。在有限维标准基下，算子半正定性与通常的矩阵二次型非负性在数学上对应；本文给出了矩阵表示接口，严格等价的 Lean 证明留作后续工作。

这一设计通过保持坐标无关性，既简化了证明，又保留了与传统表述的连接。

5.2 为何提供两个主定理版本

纸面证明中通常直接使用 ∇f 作为梯度场。在 mathlib 中，gradient f x 是一个全局定义的函数；即使缺少可微性假设，该表达式也有值。因此，在将其解释为真实梯度时，需要额外提供 HasGradientAt 或邻域内可微性条件。

若不加可微性假设而直接使用 $\text{fun } x \Rightarrow \text{gradient } f \ x$ 作为梯度场，则在该函数不可微的点上， gradient 的值不能直接解释为数学意义上的梯度。为此，本文提供了两个版本的主定理：

版本一（显式梯度场版本） 假设存在一个向量场 $g : E \rightarrow E$ ，在 x_0 的邻域上 $g \ x$ 精确等于 f 在 x 处的梯度，且 g 在 x_0 处的 Fréchet 导数为 A 。

```
theorem second_order_necessary_condition_gradient_field
  {f : E → ℝ} {g : E → E} {x₀ : E} {A : E →L[ℝ] E}
  (hmin : localMinimumAt f x₀)
  (hgrad_near : ∀f x in ℳ x₀, HasGradientAt f (g x) x)
  (hgrad_zero : g x₀ = 0)
  (hHess : HasFDerivAt g A x₀) :
  posSemidefOp A := ...
```

此版本不依赖 gradient 的全局默认值，所有梯度信息均由假设显式提供，是本文的主形式化版本。

版本二（直接 gradient 版本） 直接使用 mathlib 内置的 $\text{gradient } f$ ，并附加 f 在邻域内可微的条件。

```
theorem second_order_necessary_condition_gradient
  {f : E → ℝ} {x₀ : E} {A : E →L[ℝ] E}
  (hmin : localMinimumAt f x₀)
  (hgrad : HasGradientAt f (0 : E) x₀)
  (hdiff_near : ∀f x in ℳ x₀, DifferentiableAt ℝ f x)
  (hHess : HasFDerivAt (fun x => gradient f x) A x₀) :
  posSemidefOp A := ...
```

此版本更贴近课堂表述，但证明是通过实例化版本一完成的：在 hdiff_near 保证的邻域上， $\text{gradient } f$ 恰好充当 g 的角色。这种多版本设计既保证了逻辑严谨性，又保留了与原始数学表述的直观联系。

5.3 为何独立封装一元二阶必要条件

在纸面证明中，“一元函数在局部极小点处的二阶导数非负”常被直接引用为已知事实。但在形式化时，需要精确匹配 mathlib 中已有的分析 API。

mathlib 中已有若干一元函数极值相关定理，例如 $\text{deriv_nonneg_of_localMin}$ ；但其假设形式与本文使用的局部导数字段形式并不完全一致。本文中的一元函数 φ 的导数通过梯度与方向向量的内积给出，可导性以 $\forall^f t \text{ in } \mathcal{M} \ 0, \text{HasDerivAt } \varphi (\varphi' \ t) \ t$ （即在

0 的邻域内导数为 $\varphi'(t)$ 的形式出现。这种使用 Filter 语言描述“局部成立”的方式是 mathlib 分析学中的标准做法，但也意味着不能直接套用某一条现成定理。

因此，本文将一元二阶必要条件独立封装为引理 `oneDim_secondOrderNecessary`：

```
lemma oneDim_secondOrderNecessary
  { $\varphi$   $\varphi'$  :  $\mathbb{R} \rightarrow \mathbb{R}$ } { $l$  :  $\mathbb{R}$ }
  (hmin : localMinimumAt  $\varphi$  0)
  (hderiv : HasDerivAt  $\varphi$  0 0)
  (hderiv' : HasDerivAt  $\varphi'$   $l$  0)
  (h $\varphi'$  :  $\forall^f t$  in  $\mathcal{N}(0 : \mathbb{R}), \text{HasDerivAt } \varphi (\varphi' t) t$ ) :
   $0 \leq l := \dots$ 
```

该引理将一元部分的所有细节封装在一个模块内，使多元主定理只需调用它即可完成证明。形式化的主要困难在于将“充分小的正 t ”翻译成 $\mathcal{N}[>] 0$ 等邻域语言，并在 Lean 中处理导数符号与中值定理的组合。这种将纸面证明中隐式使用的步骤显式化为独立引理的策略，是形式化复杂分析证明时的常用方法。

5.4 证明结构的模块化分解

下表总结了数学证明中的每一步与对应 Lean 引理的关系，体现了形式化过程中的模块化分解策略。

表 2: 表 5.1 数学证明步骤与 Lean 引理对应

数学证明步骤	Lean 引理
直线 $t \mapsto x_0 + tv$ 的导数为 v	<code>hasDerivAt_line</code>
局部极小沿直线保持	<code>localMinimumAt_comp_line</code>
沿直线的一阶导数由梯度与方向的内积给出	<code>hasDerivAt_comp_line_of_hasGradientAt</code>
梯度场的导数给出 Hessian 二次型	<code>hasDerivAt_inner_gradientField_line</code>
一元二阶必要条件	<code>oneDim_secondOrderNecessary</code>
多元主定理	<code>second_order_necessary_condition_gradient_fiel</code>

这种模块化分解使每个引理专注于单一数学事实，便于独立验证和复用。主定理的证明骨架清晰地反映了数学推理的逻辑流程：任取方向 v ，构造一元函数 φ ，依次调用上述引理，最终归约到一元二阶必要条件。

6 辅助引理与证明结构

本节介绍证明主定理所需的关键引理，每个引理均先以数学语言表述，再给出对应的 Lean 陈述，并补充 $\mathbb{R}^n$ 上的详细证明。

6.1 直线限制函数

对固定的 $x_0, v \in \mathbb{R}^n$, 定义一元函数 $\varphi(t) = f(x_0 + tv)$ 。在 Lean 中直接使用 lambda 表达式即可构造该函数, 无需单独定义。

6.2 局部极小沿直线保持

引理 6.1 设 x_0 是 $f : \mathbb{R}^n \rightarrow \mathbb{R}$ 的局部极小点, 则对任意 $v \in \mathbb{R}^n$, 一元函数 $\varphi(t) = f(x_0 + tv)$ 在 $t = 0$ 处也是局部极小。

证明 由局部极小性, 存在 $\varepsilon > 0$ 使得当 $|y - x_0| < \varepsilon$ 时 $f(x_0) \leq f(y)$ 。令 $\delta = \frac{\varepsilon}{|v| + 1} > 0$ 。当 $|t| < \delta$ 时,

$$|(x_0 + tv) - x_0| = |t||v| < \delta(|v| + 1) = \varepsilon.$$

因此 $f(x_0) \leq f(x_0 + tv)$, 即 $\varphi(0) \leq \varphi(t)$, 说明 $t = 0$ 是 φ 的局部极小点。证毕。

Lean 语句:

```
lemma localMinimumAt_comp_line {f : E → ℝ} {x₀ : E}
  (hmin : localMinimumAt f x₀) (v : E) :
  localMinimumAt (fun t : ℝ => f (x₀ + t • v)) 0 := ...
```

6.3 沿直线的一阶导数与梯度

引理 6.2 设 f 在点 $x_0 + tv$ 处可微, 则

$$\varphi'(t) = \nabla f(x_0 + tv) \cdot v.$$

证明 由可微性及方向导数公式, 对于实变量函数 $\varphi(t) = f(x_0 + tv)$, 有

$$\varphi'(t) = \lim_{h \rightarrow 0} \frac{f(x_0 + tv + hv) - f(x_0 + tv)}{h} = \nabla f(x_0 + tv) \cdot v.$$

该式即为多元函数沿方向 v 的方向导数, 因可微性存在且等于梯度与方向的点积。证毕。

Lean 语句:

```
lemma hasDerivAt_comp_line_of_hasGradientAt
  {f : E → ℝ} {g : E → E} {x₀ v : E} {t : ℝ}
  (hgrad : HasGradientAt f (g (x₀ + t • v)) (x₀ + t • v)) :
  HasDerivAt (fun s : ℝ => f (x₀ + s • v))
    (inner ℝ (g (x₀ + t • v)) v) t := ...
```

6.4 Hessian 二次型的表示

引理 6.3 设梯度函数 $g(x) = \nabla f(x)$ 在 x_0 处可微, Jacobi 矩阵为 $H = H_f(x_0)$ 。定义 $\psi(t) = g(x_0 + tv) \cdot v$, 则 ψ 在 0 处可导且

$$\psi'(0) = v^\top H v.$$

证明 由 g 在 x_0 处的可微性, 有

$$g(x_0 + tv) = g(x_0) + H(tv) + o(t) = g(x_0) + tHv + o(t).$$

两边与 v 作内积, 得到

$$\psi(t) = g(x_0) \cdot v + t(Hv) \cdot v + o(t).$$

同时, $\psi(0) = g(x_0) \cdot v$ 。因此

$$\frac{\psi(t) - \psi(0)}{t} = (Hv) \cdot v + \frac{o(t)}{t}.$$

令 $t \rightarrow 0$, 得

$$\psi'(0) = (Hv) \cdot v = v^\top H v.$$

证毕。

Lean 语句:

```
lemma hasDerivAt_inner_gradientField_line
  {g : E → E} {x₀ v : E} {A : E →L[ℝ] E}
  (hHess : HasFDerivAt g A x₀) :
  HasDerivAt (fun t : ℝ => inner ℝ (g (x₀ + t • v)) v)
    (inner ℝ (A v) v) 0 := ...
```

6.5 一元二阶必要条件

引理 6.4 (一元函数的二阶必要条件) 设 $\varphi: \mathbb{R} \rightarrow \mathbb{R}$ 在 0 处局部极小, 存在 $\eta > 0$ 使得 φ 在 $(-\eta, \eta)$ 内可导, 导数记作 $\varphi'(t)$ 。若 $\varphi'(0) = 0$ 且 φ' 在 0 处可导, 则 $\varphi''(0) \geq 0$ 。

证明 记 $l = \varphi''(0) = \lim_{t \rightarrow 0} \frac{\varphi'(t) - \varphi'(0)}{t} = \lim_{t \rightarrow 0} \frac{\varphi'(t)}{t}$ 。假设 $l < 0$, 由极限的局部保号性, 存在 $\delta_1 > 0$ (可取 $\delta_1 < \eta$), 使得当 $0 < t < \delta_1$ 时 $\frac{\varphi'(t)}{t} < 0$, 故 $\varphi'(t) < 0$ 。由 φ 在 0 处局部极小, 存在 $\varepsilon > 0$, 当 $|t| < \varepsilon$ 时 $\varphi(0) \leq \varphi(t)$ 。

取 b 满足 $0 < b < \min\{\delta_1, \varepsilon\}$, 则 φ 在 $[0, b]$ 上连续, 在 $(0, b)$ 内可导。根据 Lagrange 中值定理, 存在 $c \in (0, b)$ 使得

$$\frac{\varphi(b) - \varphi(0)}{b} = \varphi'(c) < 0.$$

由此推出 $\varphi(b) < \varphi(0)$, 与 $|b| < \varepsilon$ 时的局部极小性矛盾。因此 $l \geq 0$ 。证毕。

Lean 语句:

```
lemma oneDim_secondOrderNecessary
  {φ φ' : ℝ → ℝ} {l : ℝ}
  (hmin : localMinimumAt φ 0)
  (hderiv : HasDerivAt φ 0 0)
  (hderiv' : HasDerivAt φ' l 0)
  (hφ' : ∀f t in ℵ (0 : ℝ), HasDerivAt φ (φ' t) t) :
  0 ≤ l := ...
```

通过以上五个引理, 我们已将所有必要的分析成分拆解为便于形式化的模块。主定理的证明只需将这些模块按逻辑顺序组合即可。

7 主定理的形式化证明

本节给出两个版本的定理陈述, 它们对应相同的数学内容, 只是在梯度场的处理方式上略有差异。

7.1 显式梯度场版本

下面给出省略部分技术细节后的 Lean 证明骨架:

```
theorem second_order_necessary_condition_gradient_field
  {f : E → ℝ} {g : E → E} {x₀ : E} {A : E →L[ℝ] E}
  (hmin : localMinimumAt f x₀)
  (hgrad_near : ∀f x in ℵ x₀, HasGradientAt f (g x) x)
  (hgrad_zero : g x₀ = 0)
  (hHess : HasFDerivAt g A x₀) :
  posSemidefOp A := by
  intro v
  let φ := fun t : ℝ => f (x₀ + t • v)
  let φ' := fun t : ℝ => inner ℝ (g (x₀ + t • v)) v
  have hmin_line : localMinimumAt φ 0 :=
    localMinimumAt_comp_line hmin v
  have hφ' : ∀f t in ℵ (0 : ℝ), HasDerivAt φ (φ' t) t := ...
  have hderivφ : HasDerivAt φ 0 0 := ...
  have hderivφ' : HasDerivAt φ' (inner ℝ (A v) v) 0 := ...
  exact oneDim_secondOrderNecessary hmin_line hderivφ hderivφ' hφ'
```

该版本是本文的主形式化版本, 所有梯度信息均由假设显式提供, 避免了对 `gradient` 默认值的依赖。

7.2 直接使用 gradient f 的版本

```

theorem second_order_necessary_condition_gradient
  {f : E → ℝ} {x₀ : E} {A : E →L[ℝ] E}
  (hmin : localMinimumAt f x₀)
  (hgrad : HasGradientAt f (0 : E) x₀)
  (hdiff_near : ∀f x in 𝓧 x₀, DifferentiableAt ℝ f x)
  (hHess : HasFDerivAt (fun x => gradient f x) A x₀) :
  posSemidefOp A := ...

```

此版本更贴近课堂表述，证明通过实例化版本一完成：在 `hdiff_near` 保证的邻域上，`gradient f` 充当了 `g` 的角色。

两个定理的证明结构完全遵循第 2 节所述的直线限制法：给定任意方向 v ，构造一元函数 φ ，利用引理 6.1-6.4 依次推得 $\langle Av, v \rangle \geq 0$ 。Lean 代码中的 `intro v` 即对应“任取方向”，而最终调用的 `oneDim_secondOrderNecessary` 则对应应用一元二阶必要条件。

8 与传统 Hessian 矩阵的关系

本形式化中证明的 Hessian 算子是定义在实内积空间上的连续线性映射，并不显式涉及偏导数或矩阵元素。但在有限维情形下，可以建立与传统 Hessian 矩阵的联系。我们以 n 维欧氏空间 `Euc n := EuclideanSpace ℝ (Fin n)` 为例，定义标准基下算子的矩阵表示：

```

noncomputable def hessianMatrixOfOp {n : ℕ} (A : Euc n →L[ℝ] Euc n) :
  Matrix (Fin n) (Fin n) ℝ :=
  fun i j => inner ℝ (EuclideanSpace.single i 1) (A (EuclideanSpace.single j 1))

```

此处 `EuclideanSpace.single i 1` 是第 i 个标准基向量。该矩阵的第 (i, j) 元正是 $e_i \cdot (Ae_j)$ 。若取 $A = \text{hessianOp } f x$ ，则 `hessianMatrixOfOp A` 可视为 Hessian operator 在标准基下的矩阵表示。从数学上看，标准正交基下的算子二次型 $\langle Av, v \rangle$ 与对应矩阵二次型 $v^\top H v$ 一致。

9 结论

本文利用 Lean 4 与 mathlib 数学库，完整形式化了数学分析 II 中 Hessian 半正定的二阶必要条件。我们展示了如何在 Fréchet 导数框架下重构基于偏导数的经典定理，并通过直线限制法和一元二阶必要条件给出了严格的机器检查证明。在形式化过程中，我们提供了数学概念与 Lean 代码之间的详细术语对照，系统讨论了 Hessian operator 的选择、两个主定理版本的设计、一元引理的独立封装以及证明的模块化分解等关键形

式化决策。本项目表明, Lean/mathlib 能够支持此类本科分析学命题的形式化; 但实际证明仍需要对库中的导数、梯度、邻域和中值定理接口进行细致处理。

参考文献

- [1] 华东师范大学数学科学学院. 数学分析 (下册). 高等教育出版社, 2019.
 - [2] J. Avigad, L. de Moura, S. Kong, et al.
textitTheorem Proving in Lean 4. https://leanprover.github.io/theorem_proving_in_lean4/, 2024.
 - [3] The mathlib Community. mathlib4: Lean 4 Mathematical Library. <https://github.com/leanprover-community/mathlib4>, 2024.
 - [4] The mathlib Community. Mathlib4 Documentation. https://leanprover-community.github.io/mathlib4_docs/, 2024.
 - [5] L. de Moura, S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In: Proceedings of the 28th International Conference on Automated Deduction (CADE 28), LNCS 12699, pp. 625–635, 2021.
-

A Lean 完整代码

本附录收录项目中的 Lean 源文件完整内容。

A.1 根模块 Hessian.lean

```
-- This module serves as the root of the 'Hessian' library.
-- Import modules here that should be built as part of the library.
import Hessian.Basic
```

A.2 主形式化文件 Hessian/Basic.lean

```
import Mathlib.Analysis.Calculus.Gradient.Basic
import Mathlib.Analysis.Calculus.FDeriv.Basic
import Mathlib.Analysis.Calculus.Deriv.Add
import Mathlib.Analysis.Calculus.Deriv.Comp
import Mathlib.Analysis.Calculus.Deriv.Mul
import Mathlib.Analysis.Calculus.DerivativeTest
import Mathlib.Analysis.Calculus.LocalExtr.Basic
import Mathlib.Analysis.InnerProductSpace.Basic
import Mathlib.Analysis.InnerProductSpace.Calculus

open scoped Topology
open ContinuousLinearMap
open SignType

set_option linter.unusedSectionVars false

namespace HessianPrototype

variable {E : Type*}
variable [NormedAddCommGroup E] [InnerProductSpace ℝ E] [CompleteSpace E]

/-- 'x₀' 是 'f' 的局部极小点: 在 'x₀' 的某个邻域内都有 'f x₀ ≤ f y'。 -/
def localMinimumAt (f : E → ℝ) (x₀ : E) : Prop :=
  ∃ ε > 0, ∀ y : E, dist y x₀ < ε → f x₀ ≤ f y

/-- Hessian 算子: 即梯度映射 'y ↦ gradient f y' 在 'x' 处的导数。 -/
noncomputable def hessianOp (f : E → ℝ) (x : E) : E →L[ℝ] E :=
  fderiv ℝ (fun y => gradient f y) x
```



```

/-- 连续线性算子半正定：对所有方向 'v' 都有 ' $0 \leq \text{inner } \mathbb{R} (A v) v$ '。 -/
def posSemidefOp (A : E →L[ℝ] E) : Prop :=
  ∀ v : E, 0 ≤ inner ℝ (A v) v

/-- 'f' 在 'x' 处的 Hessian 算子半正定。 -/
def hessianPosSemidefAt (f : E → ℝ) (x : E) : Prop :=
  posSemidefOp (hessianOp f x)

/-- 零算子半正定。 -/
theorem posSemidefOp_zero : posSemidefOp (0 : E →L[ℝ] E) := by
  intro v
  simp

/-- 两个半正定算子的和仍半正定。 -/
theorem posSemidefOp_add {A B : E →L[ℝ] E}
  (hA : posSemidefOp A) (hB : posSemidefOp B) :
  posSemidefOp (A + B) := by
  intro v
  dsimp [posSemidefOp] at hA hB ⊢
  have h1 := hA v
  have h2 := hB v
  calc
    0 ≤ inner ℝ (A v) v + inner ℝ (B v) v := add_nonneg h1 h2
    _ = inner ℝ (A v + B v) v := by rw [inner_add_left]
    _ = inner ℝ ((A + B) v) v := by rw [ContinuousLinearMap.add_apply]

/-- 半正定算子的非负标量倍仍半正定。 -/
theorem posSemidefOp_smul {A : E →L[ℝ] E} {c : ℝ}
  (hc : 0 ≤ c) (hA : posSemidefOp A) :
  posSemidefOp (c • A) := by
  intro v
  dsimp [posSemidefOp] at hA ⊢
  have h := hA v
  calc
    0 ≤ c * inner ℝ (A v) v := mul_nonneg hc h
    _ = (starRingEnd ℝ c) * inner ℝ (A v) v := by simp
    _ = inner ℝ (c • (A v)) v := by rw [inner_smul_left]
    _ = inner ℝ ((c • A) v) v := by rw [ContinuousLinearMap.smul_apply]

/-- 常函数的 Hessian 算子为零。 -/
theorem hessianOp_const (c : ℝ) (x : E) :

```

```

    hessianOp (fun _ : E ⇒ c) x = 0 := by
  unfold hessianOp
  simp

/-- 常函数的 Hessian 半正定。 -/
theorem hessianPosSemidefAt_const (c : ℝ) (x : E) :
  hessianPosSemidefAt (fun _ : E ⇒ c) x := by
  unfold hessianPosSemidefAt
  rw [hessianOp_const]
  exact posSemidefOp_zero

/-- 仿射直线  $t \mapsto x_0 + t \cdot v$  在任意参数  $t$  处的一维导数是方向向量  $v$ 。 -/
lemma hasDerivAt_line (x₀ v : E) (t : ℝ) :
  HasDerivAt (fun s : ℝ ⇒ x₀ + s • v) v t := by
  have hid : HasDerivAt (fun s : ℝ ⇒ s) 1 t := hasDerivAt_id' (1 : ℝ) (x := t)
  have hsmul : HasDerivAt (fun s : ℝ ⇒ s • v) v t := by
    simpa using HasDerivAt.smul_const hid v
  simpa using hsmul.const_add x₀

/-- 若  $x_0$  是  $f$  的局部极小点，则沿任意方向  $v$  的限制函数  $t \mapsto f(x_0 + t \cdot v)$  在  $0$  处有局部极小。 -/
lemma localMinimumAt_comp_line {f : E → ℝ} {x₀ : E}
  (hmin : localMinimumAt f x₀) (v : E) :
  localMinimumAt (fun t : ℝ ⇒ f (x₀ + t • v)) 0 := by
  rcases hmin with ⟨ε, hε, hmin⟩
  refine ⟨ε / (‖v‖ + 1), ?_, ?_⟩
  • positivity
  • intro t ht
  have ht_abs : |t| < ε / (‖v‖ + 1) := by
    simpa [Real.dist_eq, abs_sub_comm] using ht
  have hden_pos : 0 < ‖v‖ + 1 := by positivity
  have hmul : |t| * (‖v‖ + 1) < ε := by
    rwa [lt_div_iff₀ hden_pos] at ht_abs
  have hnorm_le : ‖t‖ * ‖v‖ ≤ ‖t‖ * (‖v‖ + 1) := by
    gcongr
    linarith [norm_nonneg v]
  have hdist : dist (x₀ + t • v) x₀ < ε := by
    calc
      dist (x₀ + t • v) x₀ = ‖t • v‖ := by
        rw [dist_eq_norm]
      congr 1

```

```

    abel
    _ = ||t|| * ||v|| := norm_smul t v
    _ ≤ ||t|| * (||v|| + 1) := hnorm_le
    _ = |t| * (||v|| + 1) := by rw [Real.norm_eq_abs]
    _ < ε := hmul
  simpa using hmin (x₀ + t • v) hdist

/-- 若 'g' 在直线点 'x₀ + t • v' 给出 'f' 的梯度，则沿直线限制函数的一维导数为 'inner ℝ
↪ (g (x₀ + t • v)) v'。 -/
lemma hasDerivAt_comp_line_of_hasGradientAt
  {f : E → ℝ} {g : E → E} {x₀ v : E} {t : ℝ}
  (hgrad : HasGradientAt f (g (x₀ + t • v)) (x₀ + t • v)) :
  HasDerivAt (fun s : ℝ ⇒ f (x₀ + s • v))
    (inner ℝ (g (x₀ + t • v)) v) t := by
  have hcomp := hgrad.hasFDerivAt.comp_hasDerivAt t (hasDerivAt_line x₀ v t)
  simpa [Function.comp_def, InnerProductSpace.toDual_apply_apply] using hcomp

/-- 若 'g' 在 'x₀' 处的 Fréchet 导数为 'A'，则 't ↦ inner ℝ (g (x₀ + t • v)) v' 在 '0'
↪ 处的导数为 'inner ℝ (A v) v'。 -/
lemma hasDerivAt_inner_gradientField_line
  {g : E → E} {x₀ v : E} {A : E →L[ℝ] E}
  (hHess : HasFDerivAt g A x₀) :
  HasDerivAt (fun t : ℝ ⇒ inner ℝ (g (x₀ + t • v)) v)
    (inner ℝ (A v) v) 0 := by
  have hgline : HasDerivAt (fun t : ℝ ⇒ g (x₀ + t • v)) (A v) 0 := by
  have hHess' : HasFDerivAt g A (x₀ + (0 : ℝ) • v) := by simpa using hHess
  simpa [Function.comp_def] using hHess'.comp_hasDerivAt 0 (hasDerivAt_line x₀ v 0)
  have hv : HasDerivAt (fun _ : ℝ ⇒ v) (0 : E) 0 := hasDerivAt_const (x := 0) (c := v)
  have hinner := hgline.inner ℝ hv
  simpa using hinner

/-- 局部极小点的一维二阶必要条件。

若 'φ' 在 '0' 处局部极小，'φ' 在 '0' 处导数为 '0'，并且某个局部导数字段
'φ'' 在 '0' 处导数为 'l'，则 '0 ≤ l'。证明思路是反证：若 'l < 0'，则
'φ'' 在 '0' 的右邻域内为负；再由中值定理得到某个足够小的 'b > 0' 满足
'φ b < φ 0'，与局部极小性矛盾。 -/
lemma oneDim_secondOrderNecessary
  {φ φ' : ℝ → ℝ} {l : ℝ}
  (hmin : localMinimumAt φ 0)
  (hderiv : HasDerivAt φ 0 0)
  (hderiv' : HasDerivAt φ' l 0)

```

```

(hφ' : ∀f t in ℳ (0 : ℝ), HasDerivAt φ (φ' t) t) :
  0 ≤ l := by
by_contra hl_nonneg
have hl_neg : l < 0 := lt_of_not_ge hl_nonneg
have hφ'_zero : φ' 0 = 0 := by
  exact (hderiv.unique hφ'.self_of_nhds).symm
have hderivφ'_eq : deriv φ' 0 = l := hderiv'.deriv
have hsign : ∀f x in ℳ (0 : ℝ), sign (φ' x) = sign (0 - x) := by
  exact eventually_nhdsWithin_sign_eq_of_deriv_neg
  (f := φ') (x₀ := 0) (by simp [hderivφ'_eq] using hl_neg) hφ'_zero
have hneg_right : ∀f x in ℳ[>] (0 : ℝ), φ' x < 0 := by
  filter_upwards [eventually_nhdsWithin_of_eventually_nhds hsign, self_mem_nhdsWithin]
    with x hxsign hxpos
  have hxsign_neg : sign (φ' x) = -1 := by
    rw [hxsign]
  have hxpos' : 0 < x := hxpos
  exact sign_neg (by linarith)
  exact sign_eq_neg_one_iff.mp hxsign_neg
have hderiv_right : ∀f x in ℳ[>] (0 : ℝ), HasDerivAt φ (φ' x) x :=
  eventually_nhdsWithin_of_eventually_nhds hφ'
rcases mem_nhdsGT_iff_exists_Ioo_subset.1 (hderiv_right.and hneg_right) with
  ⟨b₁, hb₁_pos, hb₁_subset⟩
rcases hmin with ⟨ε, hε_pos, hminε⟩
have hb₁_pos' : 0 < b₁ := hb₁_pos
let b : ℝ := min b₁ ε / 2
have hmin_pos : 0 < min b₁ ε := lt_min hb₁_pos' hε_pos
have hb_pos : 0 < b := by positivity
have hb_lt_b₁ : b < b₁ := by
  dsimp [b]
  have hmin_le : min b₁ ε ≤ b₁ := min_le_left b₁ ε
  nlinarith
have hb_lt_ε : b < ε := by
  dsimp [b]
  have hmin_le : min b₁ ε ≤ ε := min_le_right b₁ ε
  nlinarith
have hderiv_on : ∀ x ∈ Set.Ioo 0 b, HasDerivAt φ (φ' x) x := by
  intro x hx
  exact (hb₁_subset ⟨hx.1, hx.2.trans hb_lt_b₁⟩).1
have hneg_on : ∀ x ∈ Set.Ioo 0 b, φ' x < 0 := by
  intro x hx
  exact (hb₁_subset ⟨hx.1, hx.2.trans hb_lt_b₁⟩).2

```

```

have hcont : ContinuousOn  $\varphi$  (Set.Icc 0 b) := by
  intro x hx
  by_cases hx0 : x = 0
  • subst x
    exact hderiv.continuousAt.continuousWithinAt
  • have hx_pos : 0 < x := lt_of_le_of_ne hx.1 (Ne.symm hx0)
    have hx_lt_b1 : x < b1 := hx.2.trans_lt hb_lt_b1
    exact ((hb1_subset ⟨hx_pos, hx_lt_b1⟩).1).continuousAt.continuousWithinAt
rcases exists_hasDerivAt_eq_slope  $\varphi$   $\varphi'$  hb_pos hcont hderiv_on with ⟨c, hc, hc_eq⟩
have hc_neg :  $\varphi' c < 0$  := hneg_on c hc
have hslope_neg : ( $\varphi b - \varphi 0$ ) / (b - 0) < 0 := by
  simpa [hc_eq] using hc_neg
have hdiff_neg :  $\varphi b - \varphi 0 < 0$  := by
  have hbden : 0 < b - 0 := by simpa using hb_pos
  have hmul_neg : (( $\varphi b - \varphi 0$ ) / (b - 0)) * (b - 0) < 0 :=
    mul_neg_of_neg_of_pos hslope_neg hbden
  rwa [div_mul_cancel0, (ne_of_gt hbden)] at hmul_neg
have h $\varphi$ b_lt :  $\varphi b < \varphi 0$  := by linarith
have hmin_b :  $\varphi 0 \leq \varphi b$  := by
  apply hmin $\epsilon$ 
  simpa [Real.dist_eq, abs_of_pos hb_pos] using hb_lt_ $\epsilon$ 
linarith

```

-- 使用显式梯度场表述的二阶必要条件。

若 ' x_0 ' 是 ' f ' 的局部极小点, ' g ' 在 ' x_0 ' 附近给出 ' f ' 的梯度, ' $g x_0 = 0$ ',
且 ' g ' 在 ' x_0 ' 处的 *Fréchet* 导数为 ' A ', 则 ' A ' 半正定。 -/

```

theorem second_order_necessary_condition_gradient_field
  {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {g :  $\mathbb{R} \rightarrow \mathbb{R}$ } {x0 :  $\mathbb{R}$ } {A :  $\mathbb{R} \rightarrow \mathbb{R}$ }
  (hmin : localMinimumAt f x0)
  (hgrad_near :  $\forall^\mathfrak{f} x$  in  $\mathcal{N} x_0$ , HasGradientAt f (g x) x)
  (hgrad_zero : g x0 = 0)
  (hHess : HasFDerivAt g A x0) :
  posSemidefOp A := by
  intro v
  let  $\varphi : \mathbb{R} \rightarrow \mathbb{R} := \text{fun } t \Rightarrow f (x_0 + t \cdot v)$ 
  let  $\varphi' : \mathbb{R} \rightarrow \mathbb{R} := \text{fun } t \Rightarrow \text{inner } \mathbb{R} (g (x_0 + t \cdot v)) v$ 
  have hmin_line : localMinimumAt  $\varphi$  0 := localMinimumAt_comp_line hmin v
  have hline_tendsto : Filter.Tendsto (fun t :  $\mathbb{R} \Rightarrow x_0 + t \cdot v$ ) ( $\mathcal{N} (0 : \mathbb{R})$ ) ( $\mathcal{N} x_0$ ) := by
    simpa using (hasDerivAt_line x0 v 0).continuousAt.tendsto
  have hgrad_line :  $\forall^\mathfrak{f} t$  in  $\mathcal{N} (0 : \mathbb{R})$ , HasGradientAt f (g (x0 + t \cdot v)) (x0 + t \cdot v) :=

```

```

hline_tendsto.eventually hgrad_near
have hφ' : ∀f t in ℳ (0 : ℝ), HasDerivAt φ (φ' t) t := by
  filter_upwards [hgrad_line] with t ht
  exact hasDerivAt_comp_line_of_hasGradientAt (g := g) (x₀ := x₀) (v := v) ht
have hgrad_at : HasGradientAt f (g x₀) x₀ := hgrad_near.self_of_nhds
have hderivφ : HasDerivAt φ 0 0 := by
  have h := hasDerivAt_comp_line_of_hasGradientAt (g := g) (x₀ := x₀) (v := v) (t := 0)
  ↪ (by simp using hgrad_at)
  simp [φ, hgrad_zero] using h
have hderivφ' : HasDerivAt φ' (inner ℝ (A v) v) 0 := by
  simp [φ'] using hasDerivAt_inner_gradientField_line (g := g) (x₀ := x₀) (v := v)
  ↪ hHess
exact oneDim_secondOrderNecessary hmin_line hderivφ hderivφ' hφ'

/-- 直接使用 mathlib 中 'gradient f' 表述的二阶必要条件。

该版本由显式梯度场版本推出，其中梯度场取为 'fun x ⇒ gradient f x'。 -/
theorem second_order_necessary_condition_gradient
  {f : E → ℝ} {x₀ : E} {A : E →L[ℝ] E}
  (hmin : localMinimumAt f x₀)
  (hgrad : HasGradientAt f (0 : E) x₀)
  (hdiff_near : ∀f x in ℳ x₀, DifferentiableAt ℝ f x)
  (hHess : HasFDerivAt (fun x ⇒ gradient f x) A x₀) :
  posSemidefOp A := by
  apply second_order_necessary_condition_gradient_field (g := fun x ⇒ gradient f x) hmin
  • exact hdiff_near.mono fun x hx ⇒ hx.hasGradientAt
  • exact hgrad.gradient
  • exact hHess

section MatrixRepresentation

/-- 'n' 维欧氏空间。 -/
abbrev Euc (n : ℕ) := EuclideanSpace ℝ (Fin n)

/-- 连续线性算子在标准基下的矩阵表示，分量为 'inner ℝ ei (A ej)'。 -/
noncomputable def hessianMatrixOfOp {n : ℕ} (A : Euc n →L[ℝ] Euc n) :
  Matrix (Fin n) (Fin n) ℝ :=
  fun i j ⇒ inner ℝ (EuclideanSpace.single i 1) (A (EuclideanSpace.single j 1))

end MatrixRepresentation

```

```
end HessianPrototype
```